

csrf, errorHandler, express-session, method-override, morgan, response-time, serve-favicon, serve-index, serve-static, vhost Error-Middleware: bearbeitet Errors, welche von den MWs generiert wurden, sollte zuletzt registriert werden, mehrere sind möglich, wenn next() mit einem Error-Objekt aufrufen-fen wird, werden keine MWs mehr ausgeführt und stattdessen die Error-MW ausgeführt, 4 Params: Body-parser: parst Request-Bodies und stellt die Variablen unter req.body zur Verfügung Cookie-parser: Parst den Cookie Header und stellt die Cookies unter req.cookies zur Verfügung Express-session: req.session ist ein Objekt, das temporär Infos über aktuelle User Sessions speichert Error-handler: development-only, schickt error stack traces zurück zum Client je nach Konfiguration Csurf: verhindert Cross-Site Request Forgery (CSRF) Angriffe durch CSRF Tokens in Cookies express.static: Bereitstellung statischer Files wie Bilder, CSS-/JS-Files, Beispiel am Anfang des Kapitels express.router: Router fürs Request-Handling, Beispiel router const und use(router) am Anfang des Kapitels Resquest-Parameter: Ressourcenidentifikation, Abbildung hierarchischer Datenstrukturen URL Query Parameter: Such-/Filter-Operationen, Steuerung Pagination, Beispiel für <search? t1=&t2=b> NEDB: dokumentenorientierte, file-backed Datenbank Cookies: im Browser, können persistent sein, automatisch bei jeder Anfrage gesendet, für Sitzungsverwaltung, Personalisierung, Tracking und Analyse Session Objekte: auf dem Server, Lebensdauer bis Ablauf oder Logout, nur innerhalb von Express verfügbar, für Sitzungsverwaltung und speichern temporärer Infos über aktuelle User Sessions REST: Verschiedene Paths für verschiedene Ressourcen, HTTP-Verben & JSON korrekt verwenden, stateless: Session durch JWT Token im Header ersetzen Neben json(): arrayBuffer(), blob(), formData() & text() Web-Sockets: bidirektionale, dauerhafte Kommunikations-Verbindung zwischen Client & Server, geringe Latenz, wenig Overhead, komplexe API (socket.io), für Echtzeit-Apps: Chat, Game, Live-Ticker, Docs-Collab, Push-Nachrichten, interaktive Video- / Audio-Streams Handlebars.js: Template Engine, um auf der Server-Side HTML Code zu generieren

TS
TypeScript-ESLint: statische Analyse des Codes zur Fehlerfindung, Installation: "npm install --save-dev eslint @eslint/js @types/eslint__js typescript typescript-eslint", Lint ausführen: <code>npm eslint .</code>, eslint.config.js

Prettier: formatiert Code beim Speichern, setzt konsistente Stilregeln durch, ESLint berücksichtigt auch Prettier Regeln, VS Code Extension, prettierrc: Basis-Typen: boolean, number, string, null, undefined (no value), any (don't care), unknown (don't know), void, never, array, tuple, enum, union

```
declare let v3: string; let v1 = «Hayes»; let v2 : number; v2 = 1; interface User { name: string; readonly id: number; } class UserAccount implements User { constructor( public name: string, public id: number ) {} const user: User = { name: v1, id: v2 }; const user = { name: v1, id: v2 }; function deleteUser(user: User) { ... } function getAdminUser(): User { ... } type WindowStates = «open» | «closed» | «minimized»; type WindowStatesLong = ${WindowStates} -window; let myInferredNumArray = [1, 2, 3]; let myNumArray: Array = [1, 2, 3]; let myTuple: [number, string] = [1, «abcd»]; let unknownVar: unknown; let numberVar = 1; unknownVar = 1; if typeof unknownVar === «number») numberVar = unknownVar function add(s1: string, s2: string): string; function add(n1: number, n2: number): number; function combineFunction(sn: number | string = «», ns?: number): string { return String(sn)+String(Ins |) «»); } function numberApplicator(numArray: number[], numFun: (prevRes: number, current: number) => number): number { return numArray.reduce(numFun); } interface PointList { itemLIst: T[]; } const myCanvas = document.getElementById(«main_canvas») as HTMLCanvasElement;
```

ACCESSIBILITY
Rechtlich: UNO-Behindertenrechtskonvention (UNO-BRK), gleichberechtigte Möglichkeiten im Zugang & Benutzung von Informations- & Kommunikationssystemen Häufigste Fehler: Text zu wenig Kontrast, kein alt Text, leere Links/Buttons/Doc-Sprache, fehlende Form Labels Farbenblindheit: prüfen mit Dev-Tools (Rendering), Doppel-Codierung notwendig (Farbe & Form/icon/Text), kritische Informationen nicht nur mit Text codieren, farbenblind-freundliche Farbpaletten wählen Farbkontrast: prüfen mit Dev-Tools (Color Picker) & Lighthouse / Firefox Report, Zielgruppe erweitern mit WCAG Kontrast Level AA (5.7:1, 50%), AAA (15.9:1, 80%) Zoombarkeit: Zoom nicht unterbinden, Falsch: user-scalable=0/no, maximum-scale=1 Animationen: sollten abstellbar sein, Verringerung der Ablenkung bei Konzentrationsstörung, Verhinderung der Auslösung von Epilepsie und Migräne Tastatur-Bedienbarkeit: alle wichtigen Input-Elemente sind in der richtigen Reihenfolge fokussierbar, verboten: float, flex-direction: reverse, **Standard Input Controls nutzen: a, button, input[type=«...»], textarea, zusätzliche Elemente können mit «tabindex=0» fokussierbar gemacht werden, Elemente jeder Seite können mit «autofocus» versehen werden Screenreader-Optimierung: keine Headings Level auslassen, semantische Elemente richtig nutzen, Skip-Links am Anfang der Site, lang Attribut korrekt setzen ARIA Attribute: lösen Accessibility Issues, die nicht mit nativem HTML gelöst werden können, heissen jeweils aria-, wichtigste: hidden, label, expanded, current, labelledby, controls, haspopup, invalid, live, required Tabellen: Heading für Rows & Columns mit «th» ausstaten, alternativ <td role=«columnheader/rowheader"> Allgemeines: alt Tag bei Bildern, leer wenn dekorativ, logische Reihenfolge auch ohne CSS, semantische Struktur, Multimedia: 2-Sinne Prinzip, Anzeige-Art kann angepasst werden (Textgrösse, Animationen, Zeitlimits), Überschriften/Form Labels/Linktexte sind verständlich, auf Formatwechsel (z.B. PDF) wird hingewiesen, konsistente Navigation, valides HTML kompatibel mit versch. I/O Geräten, Hilfestellung bei Interaktionen, PDFs auf PDF**

SECURITY
Stored XSS: Server kann so manipuliert werden, dass JS Code an die Browser der Opfer ausgeliefert & ausgeführt wird, Massnahmen: User-Input muss beim Output encodet werden, z.B. mit 3x () in Handlebars, oder User-Input wird sanitized, z.B. mit xss oder dompurify Library JS Injection: dem Server kann JS Code gesendet werden, welcher ihn im Code mit eval()/setTimeout()/setInterval()/Function() ausführt, Massnahmen: stattdessen parseXXX()/JSON.parse() verwenden, globale Scopes & Variablen reduzieren, rechen-intensive Tasks mit childprocess.spawn auslagern, Node kein Root-Prozess Insecure direct object references: durch Manipulation der URL lassen sich sensible Daten ohne Authentifizierung oder Autorisierung aufrufen, Massnahmen: bei allen Sites mit sensiblen Daten sicherstellen, dass der User eingeloggt und berechtigt ist

CSRF
Cross Site Request Forgery: Angreifer bringt Nutzer mit gültiger Session dazu, auf einer Site ein gefälschtes Formular an die richtige Site zu submitten, Massnahmen: Formular-Submission-Requests (GET/POST) müssen geprüft werden, ob sie von einem vom Server frisch ausgelieferten Formular stammen (mit CSRF Token) Replay attacks: z.B. bei Quiz wird mit Formular Aufgabe und User-Input-Lösung submitted, wenn richtig gibt es einen Punkt, das kann replayed werden, sollte aber nicht gehen, da der User dann unrechtmässig Punkte erhält, Massnahmen: CSRF Token 1x gültig, bereits gelöste Aufgaben speichern, letzte Aufgabe in Session speichern Cryptographic failures: Crypto schlecht eingesetzt, Massnahmen: https nutzen, keine geheimen Infos in Query-Params (Leaks in Server Logs oder Browser History), Authentication Service nutzen Identification & Authentication failures: z.B. in Session Timeout zu lang, User bleibt auf öffentlichem PC eingeloggt, Massnahme: Session Timeout sinnvoll setzen CORS Header: ermöglicht Laden von Ressourcen (Bilder, Scripts, usw.) von anderem Ursprungsort als eigener Origin, man beschränkt, welche Websites auf seine Res-sourcen zugreifen dürfen

CSP Header: Content Security Policy, Beispiel schränkt Kommunikation auf Source Domain/api.ch ein, externe & Inline-Scripts sind verboten, aktivieren mit unsafe-inline Weiteres: DNS Prefetching & Client Caching deaktivieren, Public Key Pinning Headers, Strict-Transport-Security Header (HSTS) setzen, X-XSS-Protection Header, Safe-Regex verwenden (DDoS), Parameter pollution Cookies: Secure Attribut: Cookie wird nur über HTTPS versendet, nicht HTTP (ausser localhost), HttpOnly Attribut: deaktiviert Zugriff auf Cookies aus JS